Pre-Assessment: Q1–Q15

Q1: What is R and what is RStudio? How do the two relate to each other?

R is the underlying programming language and software environment used for statistical computing, data analysis, and graphics. It is the actual "engine" that processes your data and runs calculations.

RStudio is an Integrated Development Environment (IDE) built specifically for R. It serves as the visual dashboard or "cockpit," providing a user-friendly interface with panels for writing scripts, viewing plots, tracking active variables, and organizing files.

Relationship: R works completely fine on its own via a command-line terminal, but RStudio cannot function without R. RStudio simply makes interacting with the R engine much more efficient and visual.

Q2: Why does an analyst need a script rather than typing commands one by one in the console?

- **Reproducibility:** A script saves every step of your data workflow, allowing you, or a colleague, to rerun the entire analysis from scratch and get identical results.
- **Automation:** If your raw data changes (e.g., new survey results arrive), you can refresh your entire analysis instantly by running the script instead of re-typing dozens of lines.
- **Error Correction:** If you make a mistake on step 2 of a 50-step analysis, a script allows you to fix that single line and execute the rest cleanly, rather than manually re-entering 48 subsequent commands.

Q3: Explain the difference between numeric, character, logical, and factor values in R. Give one example of each from survey data.

- **Numeric:** Quantitative values used for continuous or discrete mathematical measurements.
  Survey Example: Household size (5) or total livestock owned (42).
- **Character:** Text strings used to represent names, descriptive labels, or open-ended responses.
  Survey Example: Name of the respondent ("Amina") or open feedback ("Highly satisfied with the veterinary extension services").
- **Logical:** Boolean data that can only take one of two absolute values: TRUE or FALSE. They are often the result of conditional testing.
  Survey Example: Has electricity? (TRUE / FALSE).
- **Factor:** Categorical data that looks like text but is stored internally as integers with predefined levels. Factors are crucial for sorting categorical data properly in statistical modeling and charts.
  Survey Example: Education level ("None", "Primary", "Secondary", "Higher").

Q4: What does the assignment operator <- do? Write one line that stores the value 2016 in an object called year.

Function: The assignment operator takes the value or output on its right side and stores it inside the variable/object name designated on its left side.

```
year <- 2016
```

Q5: A colleague has a vector of region codes 1, 2, 3. How would you attach the labels 'Afar', 'Amhara', 'Oromia' to them?

You can convert the numeric vector into a factor and explicitly supply the corresponding labels.

```
# Assuming the vector is named region_codes
region_factor <- factor(region_codes,
                        levels = c(1, 2, 3),
                        labels = c("Afar", "Amhara", "Oromia"))
```

Q6: Describe what a data frame is and how it differs from a matrix.

Data Frame: A two-dimensional, spreadsheet-like structural layout where rows represent observations and columns represent variables. Crucially, a data frame is *heterogeneous*, meaning different columns can hold different types of data (e.g., Column 1 is numeric, Column 2 is character, Column 3 is logical).

Difference from a Matrix: While a matrix is also two-dimensional, it is strictly *homogeneous*. Every single cell in a matrix must contain the exact same data type (e.g., all numbers). If you try to mix numbers and text in a matrix, R will forcefully convert all numbers into text.

Q7: Write, in words or code, how you would read a CSV file named livestock.csv into R.

In words: You use R's built-in `read.csv()` function (or the faster `read_csv()` function from the `readr` package), passing the file path inside quotation marks, and assigning the result to an object.

```
livestock_data <- read.csv("livestock.csv")
```

Q8: What do head(), str(), and summary() tell you about a dataset? When would you use each?

Q9: A column contains missing values. What happens when you call mean() on it, and how do you fix it?

What happens: R will return `NA` as the result. R does this safely by default because it assumes that if any data point is missing, the mathematical mean is fundamentally unknown.

How to fix it: Add the argument `na.rm = TRUE` inside the function to explicitly instruct R to strip out the missing values before executing the calculation.

```
mean(dataset$column_name, na.rm = TRUE)
```

Function	What it tells you	When to use it
`head()`	Displays the first few rows (usually 6) of your dataset.	Use it right after loading data to visually confirm that columns and values imported cleanly.
`str()`	Shows the structural layout of the dataset, including dimensions, column names, and data types (int, num, chr, Factor).	Use it to diagnose data structure issues (e.g., ensuring a date column isn't accidentally stored as text).
`summary()`	Computes descriptive statistics for each column (Min, Max, Mean, Median for numeric columns; frequency counts for categorical ones).	Use it during early exploratory analysis to spot outliers, check data ranges, or identify missing data.

Q10: How would you keep only the rows of a data frame where Region equals 'Oromiya' and year is greater than 2016?

Using the `filter()` function from the `dplyr` package:

```
filtered_data <- filter(my_dataframe, Region == "Oromiya" & year > 2016)
```

(Note: If using base R, you would use: `my_dataframe[my_dataframe$Region == "Oromiya" & my_dataframe$year > 2016, ]`)

Q11: Name three dplyr verbs and describe what each one does.

- `filter()`: Subsets rows based on specified logical conditions (keeps observations that meet your criteria).
- `select()`: Subsets columns by their names (keeps or drops specific variables).
- `mutate()`: Creates new columns or modifies existing ones by performing calculations on other columns.

Q12: Explain what the pipe %¿% does and why analysts like it.

What it does: The pipe operator takes the output of the expression on its left side and automatically feeds it forward as the first argument of the function on its right side.

Why analysts like it: It eliminates the need to create cluttered, temporary intermediate variables or write hard-to-read, nested code (functions within functions). It allows analysts to write data cleaning workflows sequentially, reading from left-to-right and top-to-bottom like a human recipe book.

Q13: In ggplot2, what is an aesthetic and what is a geom? Give an example of each.

- **Aesthetic (aes):** Defines the structural mapping between variables in your dataset and visual properties on the plot. It dictates what data connects to what visual element.
  Example: Mapping household income to the x axis, or mapping region type to the color.
- **Geom (geom_...):** Represents the actual geometric object or shape used to render that data on the canvas. It determines how the chart looks.
  Example: `geom_point()` for a scatter plot, or `geom_bar()` for a bar chart.

Q14: You have two files that share a zone column. How would you combine them into one table, and what could go wrong?

How to combine: You can perform a mutating join operation (such as `left_join()` or `inner_join()` from `dplyr`), specifying the zone column as the merging key.

```
combined_table <- left_join(file_A, file_B, by = "zone")
```

What could go wrong:

- **Mismatched Text:** Typos or capitalization mismatches (e.g., "Zone 1" vs "zone 1 " with trailing spaces) will prevent rows from matching up.
- **Many-to-Many Duplication:** If the zone key is duplicated multiple times in both tables, the join can accidentally create massive Cartesian product combinations, artificially inflating your row counts.
- **Incompatible Data Types:** If one file stores zone as a numeric integer (e.g., 1) and the other stores it as a character text string (e.g., "1"), R will throw an error and refuse to merge them until converted.

Q15: What is a dashboard, and what is the difference between a report and a dashboard?

Dashboard: A visual tool that curates, consolidates, and displays key metrics, charts, and key performance indicators (KPIs) on a single screen, often updating in real-time or allowing interactive user filtering.

The Difference:

- **Reports** are static, document-driven, and linear. They provide deep, highly detailed, text-heavy explanations of data at a specific, frozen point in time, usually formatted for reading (like a PDF).
- **Dashboards** are dynamic, high-level, and visual. They are built for quick, scanning monitoring, allowing stakeholders to actively interact with filters to answer their own immediate questions.

Would you like me to clarify the syntax for any of these R functions, or would you like to run a quick mock scenario based on one of the questions to practice your skills before the training session?